

Tuples: Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data Structures. **Lists:** List values, Accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, Cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list and range, Nested lists, Matrices. **Dictionaries:** Dictionary operations, dictionary methods, aliasing and copying. **Functions:** Functions with arguments and return values, Lambda Functions.

Lists

Motivation for using Lists

- ✓ We use lists in our everyday lives—we make **shopping lists, to-do lists, and mental checklists**.
- ✓ **Our DNA is essentially a long list of molecules** in the form of a double helix, found in the nucleus of all human cells and all living organisms.
- ✓ Its purpose is also to **store information**—specifically, the instructions that are used to construct all other cells in the body—that we call *genes*.
- ✓ *Given the 2.85 billion nucleotides that make up the human genome, determining their sequencing (and thus understanding our genetic makeup) is fundamentally a computational problem.*

List Characteristics

List Characteristics	Elements
Element Type	All elements of the same type
	Elements of different types
Length	Fixed length
	Varying length
Modifiability	Mutable (alterable)
	Immutable (unalterable)
Common Operations	Determine if a list is empty
	Determine the length of a list
	Access (retrieve) elements of a list
	Insert elements into a list
	Replace elements of a list
	Delete elements of a list
	Append elements to (the end of) a list

What are Lists in Python?

- A list is a **linear data structure**, meaning that its elements have a linear ordering.
- That is, there is a first element, a second element, and so on.
- The **values** that make up a list are called its **elements**, or its items

0:	68.8
1:	70.2
2:	67.2
3:	71.8
4:	73.2
5:	75.6
6:	74.0

How to create Lists in Python

```
>>> colors = ['red', 'blue', 'green']
```

```
>>> colors
```

Output:

```
['red', 'blue', 'green']
```

Use square brackets `[]` to create a list

```
colors=['red','blue','green',12,13,45,4.5,6.78]
print(colors)
print(type(colors))

['red', 'blue', 'green', 12, 13, 45, 4.5, 6.78]
<class 'list'>
```

```

type_x=colors[0]
print(type_x)
type(type_x)

red

str

print(colors)

['red', 'blue', 'green', 12, 13, 45, 4.5, 6.78]

print(colors[0])
print(colors[0:5])
print(colors[0:5:2])

red
['red', 'blue', 'green', 12, 13]
['red', 'green', 13]

```

Accessing values in a Lists

```

#Use slicing operator and index value
colors=['red','blue','green']
print(colors[1])
print(colors[0:2])
print(colors[:])
print(colors[-1])

blue
['red', 'blue']
['red', 'blue', 'green']
green

```

Updating values in a List: Lists are mutable

```

colors=['red','blue','green']
print(colors)
#Assign string
colors[1]='yellow'
print("After updating the list value",colors)
#Assign integer
colors[2]=5
print("Showcasing lists can have different data type values",colors)
#Update through slicing operator
colors[0:2]=[1,2]
print("Updation through list slicing operation",colors)

['red', 'blue', 'green']
After updating the list value ['red', 'yellow', 'green']

```

Showcasing lists can have different data type values ['red', 'yellow', 5]

Updation through list slicing operation [1, 2, 5]

```
x=[1,1,2,3,4,5]
```

```
print(x)
```

```
print(x.count(1))
```

```
[1, 1, 2, 3, 4, 5]
```

```
2
```

```
x.append(10)
```

```
print(x)
```

```
[1, 1, 2, 3, 4, 5, 10]
```

```
x.append([10,20])
```

```
print(x)
```

```
print(x[7])
```

```
print(x[7][0])
```

```
print(x[7][1])
```

```
[1, 1, 2, 3, 4, 5, 10, [10, 20], [10, 20], [10, 20]]
```

```
[10, 20]
```

```
10
```

```
20
```

```
x=[1,2,3,4,5]
```

```
print(x)
```

```
x.extend([10,20])
```

```
print(x)
```

```
[1, 2, 3, 4, 5]
```

```
[1, 2, 3, 4, 5, 10, 20]
```

```
x=[1,2,3,4,5]
```

```
x.append(3)
```

```
print(x)
```

```
x.append([7,8])
```

```
print(x)
```

```
x.extend([3,4,5,6])
```

```
print(x)
```

```
print(x.pop())
```

```
print(x)
```

```
[1, 2, 3, 4, 5, 3]
```

```
[1, 2, 3, 4, 5, 3, [7, 8]]
```

```
[1, 2, 3, 4, 5, 3, [7, 8], 3, 4, 5, 6]
```

```
6
```

```
[1, 2, 3, 4, 5, 3, [7, 8], 3, 4, 5]
```

```
print(x)
```

```
[1, 2, 3, 4, 5, 3, [7, 8], 3, 4, 5]
x.insert(0, 'HelloPython')
print(x)
x[0]="Replace"
print(x)

['HelloPython', 'HelloPython', 1, 2, 3, 4, 5, 3, [7, 8], 3, 4, 5]
['Replace', 'HelloPython', 1, 2, 3, 4, 5, 3, [7, 8], 3, 4, 5]

x.remove(1)
print(x)
x.remove(3)
print(x)

['Replace', 'HelloPython', 2, 3, 4, 5, 3, [7, 8], 3, 4, 5]
['Replace', 'HelloPython', 2, 4, 5, 3, [7, 8], 3, 4, 5]

print(x)

['HelloPython', 2, 4, 5, 3, [7, 8], 3, 4, 5]

help(x)
```

Help on list object:

```
class list(object)
| list(iterable=(), /)
|
| Built-in mutable sequence.
|
| If no argument is given, the constructor creates a new empty list.
| The argument must be an iterable if specified.
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| __contains__(self, key, /)
|     Return key in self.
|
| __delitem__(self, key, /)
|     Delete self[key].
|
| __eq__(self, value, /)
|     Return self==value.
|
| __ge__(self, value, /)
|     Return self>=value.
```

```

__getattr__(self, name, /)
    Return getattr(self, name).

__getitem__(...)
    x.__getitem__(y) <==> x[y]

__gt__(self, value, /)
    Return self>value.

__iadd__(self, value, /)
    Implement self+=value.

__imul__(self, value, /)
    Implement self*=value.

__init__(self, /, *args, **kwargs)
    Initialize self. See help(type(self)) for accurate signature.

__iter__(self, /)
    Implement iter(self).

__le__(self, value, /)
    Return self<=value.

__len__(self, /)
    Return len(self).

__lt__(self, value, /)
    Return self<value.

__mul__(self, value, /)
    Return self*value.

__ne__(self, value, /)
    Return self!=value.

__repr__(self, /)
    Return repr(self).

__reversed__(self, /)
    Return a reverse iterator over the list.

__rmul__(self, value, /)
    Return value*self.

__setitem__(self, key, value, /)
    Set self[key] to value.

__sizeof__(self, /)

```

Return the size of the list in memory, in bytes.

`append(self, object, /)`
Append object to the end of the list.

`clear(self, /)`
Remove all items from list.

`copy(self, /)`
Return a shallow copy of the list.

`count(self, value, /)`
Return number of occurrences of value.

`extend(self, iterable, /)`
Extend list by appending elements from the iterable.

`index(self, value, start=0, stop=9223372036854775807, /)`
Return first index of value.

Raises `ValueError` if the value is not present.

`insert(self, index, object, /)`
Insert object before index.

`pop(self, index=-1, /)`
Remove and return item at index (default last).

Raises `IndexError` if list is empty or index is out of range.

`remove(self, value, /)`
Remove first occurrence of value.

Raises `ValueError` if the value is not present.

`reverse(self, /)`
Reverse **IN PLACE**.

`sort(self, /, *, key=None, reverse=False)`
Stable sort **IN PLACE**.

Static methods defined here:

`__new__(*args, **kwargs) from builtins.type`
Create and return a new object. See `help(type)` for accurate signature.

Data and other attributes defined here:

__hash__ = None

List Methods

Method	Description
L.append(v)	Appends value v to list L
L.clear()	Removes all items from list L
L.count(v)	Returns the number of occurrences of v in list L
L.extend(v)	Appends the items in v to L
L.index(v)	Returns the index of the first occurrence of v in L—an error is raised if v doesn't occur in L.
L.index(v, beg)	Returns the index of the first occurrence of v at or after index beg in L—an error is raised if v doesn't occur in that part of L.
L.index(v, beg, end)	Returns the index of the first occurrence of v between indices beg (inclusive) and end (exclusive) in L; an error is raised if v doesn't occur in that part of L.
L.insert(i, v)	Inserts value v at index i in list L, shifting subsequent items to make room
L.pop()	Removes and returns the last item of L (which must be nonempty)
L.remove(v)	Removes the first occurrence of value v from list L
L.reverse()	Reverses the order of the values in list L
L.sort()	Sorts the values in list L in ascending order (for strings with the same letter case, it sorts in alphabetical order)
L.sort(reverse=True)	Sorts the values in list L in descending order (for strings with the same letter case, it sorts in reverse alphabetical order)

```
x=[1,2,0,7,5,6]
print(x)

[1, 2, 0, 7, 5, 6]
x.sort(reverse=True)
print(x)

[7, 6, 5, 2, 1, 0]
None

print(x)
print(x.reverse())

[0, 1, 2, 5, 6, 7]
None
```

help(x)

Help on list object:

```
class list(object)
| list(iterable=(), /)
|
| Built-in mutable sequence.
|
| If no argument is given, the constructor creates a new empty list.
| The argument must be an iterable if specified.
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| __contains__(self, key, /)
|     Return key in self.
|
| __delitem__(self, key, /)
|     Delete self[key].
|
| __eq__(self, value, /)
|     Return self==value.
|
| __ge__(self, value, /)
|     Return self>=value.
|
| __getattr__(self, name, /)
|     Return getattr(self, name).
|
| __getitem__(...)
|     x.__getitem__(y) <==> x[y]
|
| __gt__(self, value, /)
|     Return self>value.
|
| __iadd__(self, value, /)
|     Implement self+=value.
|
| __imul__(self, value, /)
|     Implement self*=value.
|
| __init__(self, /, *args, **kwargs)
|     Initialize self. See help(type(self)) for accurate signature.
|
| __iter__(self, /)
|     Implement iter(self).
```

```

__le__(self, value, /)
    Return self<=value.

__len__(self, /)
    Return len(self).

__lt__(self, value, /)
    Return self<value.

__mul__(self, value, /)
    Return self*value.

__ne__(self, value, /)
    Return self!=value.

__repr__(self, /)
    Return repr(self).

__reversed__(self, /)
    Return a reverse iterator over the list.

__rmul__(self, value, /)
    Return value*self.

__setitem__(self, key, value, /)
    Set self[key] to value.

__sizeof__(self, /)
    Return the size of the list in memory, in bytes.

append(self, object, /)
    Append object to the end of the list.

clear(self, /)
    Remove all items from list.

copy(self, /)
    Return a shallow copy of the list.

count(self, value, /)
    Return number of occurrences of value.

extend(self, iterable, /)
    Extend list by appending elements from the iterable.

index(self, value, start=0, stop=9223372036854775807, /)
    Return first index of value.

    Raises ValueError if the value is not present.

```

`insert(self, index, object, /)`
Insert object before index.

`pop(self, index=-1, /)`
Remove and return item at index (default last).

Raises `IndexError` if list is empty or index is out of range.

`remove(self, value, /)`
Remove first occurrence of value.

Raises `ValueError` if the value is not present.

`reverse(self, /)`
Reverse `*IN PLACE*`.

`sort(self, /, *, key=None, reverse=False)`
Stable sort `*IN PLACE*`.

Static methods defined here:

`__new__(*args, **kwargs) from builtins.type`
Create and return a new object. See `help(type)` for accurate signature.

Data and other attributes defined here:

`__hash__ = None`

To create a List

- The simplest is to enclose the elements in square brackets **[and]**:

```
ps = [10, 20, 30, 40]
qs = ["spam", "bungee", "swallow"]
```

- The elements of a list don't have to be the same type.

```
zs = ["hello", 2.0, 5, [10, 20]]
an_empty_list = []
```

List Methods Example

Operation	fruit = ['banana', 'apple', 'cherry']	
Replace	fruit[2] = 'coconut'	['banana', 'apple', 'coconut']
Delete	del fruit[1]	['banana', 'cherry']
Insert	fruit.insert(2, 'pear')	['banana', 'apple', 'pear', 'cherry']
Append	fruit.append('peach')	['banana', 'apple', 'cherry', 'peach']
Sort	fruit.sort()	['apple', 'banana', 'cherry']
Reverse	fruit.reverse()	['cherry', 'banana', 'apple']

Answer this?

LET'S TRY IT

From the Python Shell, enter the following and observe the results.

>>> lst = [10, 20, 30]	>>> del lst[2]
>>> lst	>>> lst
???	???
>>> lst[0]	>>> lst.insert(1, 15)
???	>>> lst
	???
>>> lst[0] = 5	>>> lst.append(40)
>>> lst	>>> lst
???	???

```
lst=[10,20,30]
print(lst)
print(lst[0])
lst[0]=5
print(lst)
del lst[2]
print(lst)
lst.insert(1,15)
print(lst)
lst.append(40)
print(lst)

[10, 20, 30]
10
[5, 20, 30]
[5, 20]
[5, 15, 20]
[5, 15, 20, 40]
```

append vs extend methods in Lists

- nums.append([6,7,8])

Initial list is

[1, 2, 3, 4, 5]

After appending

[1, 2, 3, 4, 5, [6, 7, 8]]

- nums.extend([6,7,8])

Initial list is

[1, 2, 3, 4, 5]

After extending

[1, 2, 3, 4, 5, 6, 7, 8]

- **“extend”** takes a single argument, which is always a list, and adds each of the elements of that list to the original list.
- **“append”** takes one argument, which can be any data type, and simply **adds it to the end of the list**.

```
lst=[1,2,3,4,5]
lst.append([6,7,8])
print(lst)
lst.extend([9,10,11])
print(lst)
#inner list should be accessed as shown
print(lst[5])
print(lst[5][2])

[1, 2, 3, 4, 5, [6, 7, 8]]
[1, 2, 3, 4, 5, [6, 7, 8], 9, 10, 11]
[6, 7, 8]
8
```

List Concatenation

The + operator concatenates lists:

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> c
[1, 2, 3, 4, 5, 6]
```

Similarly, the * operator repeats a list a given number of times:

```
>>> [0] * 4
[0, 0, 0, 0]
>>> [1, 2, 3] * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

The first example repeats [0] four times. The second example repeats the list [1, 2, 3] three times.

```
a=[1,2,3]
b=[4,5,6]
#Concatenation of 2 different lists
c=a+b
print(a)
print(c)
#Repeat given numbers in a list many times
print([0]*5)
print(c*3)

[1, 2, 3]
[1, 2, 3, 4, 5, 6]
[0, 0, 0, 0, 0]
[1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6]

a=[1,2,3]
b=[1,2,3,4,5]
c=a+b
print(c)

[1, 2, 3, 1, 2, 3, 4, 5]
```


List Slices

- **a[start:end]**
 - # items start through end-1
- **a[start:]**
 - # items start through the rest of the array
- **a[:end]**
 - # items from the beginning through end-1
- **a[:]**
 - # a copy of the whole array
- **a[start:end:step]**
 - # start through not past end, by step

```
lst=['a','b',5,7,8,"Hello","World"]
print(lst[0:6])
print(lst[0:])
print(lst[:6])
print(lst[:])
print(lst[0:7:2])
print(lst[-1])
print(lst[-4:-1])
print(lst[-6:-1:2])

['a', 'b', 5, 7, 8, 'Hello']
['a', 'b', 5, 7, 8, 'Hello', 'World']
['a', 'b', 5, 7, 8, 'Hello']
['a', 'b', 5, 7, 8, 'Hello', 'World']
['a', 5, 8, 'World']
World
[7, 8, 'Hello']
['b', 7, 'Hello']

x=[1,2,3,4,5]
print(x)
x.reverse()
print(x)
```

```
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]
```

Deleting List Elements

```
a=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18]
del a[0]
print(a)
del a[0:6:2]
print(a)
del a[0:4]
print(a)
del a[:]
print(a)
a.clear()
print(a)
del a
print(a)
```

```
[2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18]
[3, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18]
[9, 10, 11, 12, 13, 14, 15, 16, 17, 18]
[]
[]
```

```
-----
-----
NameError                                Traceback (most recent call
last)
<ipython-input-36-4a5e32289211> in <module>
      11 print(a)
      12 del a
----> 13 print(a)

NameError: name 'a' is not defined
```

Removing and searching List elements

- **“remove”** removes the first occurrence of a value from a list.
- If the value is not found in the list it raises an exception.
- **Pop does two things:** it removes the last element of the list, and it returns the value that it removed. Note that this is different from `li[-1]`, which returns a value but does not change the list, and different from `li.remove(value)`, which changes the list but does not return a value.

```
li = ['a', 'b', 'new', 'mpilgrim', 'z', 'example', 'new', 'two', 'elements']
li.remove("z")
print(li)
li.remove("new")
print(li)
print(li.pop())
li.index("example")
print("c" in li)
li.remove("c")
li.index("c")
```

```
['a', 'b', 'new', 'mpilgrim', 'example', 'new', 'two', 'elements']
['a', 'b', 'mpilgrim', 'example', 'new', 'two', 'elements']
elements
False
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
<ipython-input-37-086fb2d30127> in <module>
      7 li.index("example")
      8 print("c" in li)
----> 9 li.remove("c")
```

```
10 li.index("c")
11
```

ValueError: list.remove(x): x not in list

```
x=[1,2,3,4,5]
print(x.pop())
print(x)
print(x.pop())
print(x)
x.remove(100)
print(x)
```

```
5
[1, 2, 3, 4]
4
[1, 2, 3]
```

```
-----
-----
ValueError                                Traceback (most recent call
last)
<ipython-input-36-8cd91c40c8cf> in <module>
      4 print(x.pop())
      5 print(x)
----> 6 x.remove(100)
      7 print(x)
```

ValueError: list.remove(x): x not in list

Built in List methods

[cmp\(list1, list2\)](#)

Compares elements of both lists.

[len\(list\)](#)

Gives the total length of the list.

[max\(list\)](#)

Returns item from the list with max value.

[min\(list\)](#)

Returns item from the list with min value.

[list\(seq\)](#)

Converts a tuple into list.

[list.append\(obj\)](#)

Appends object obj to list

[list.count\(obj\)](#)

Returns count of how many times obj occurs in list

[list.extend\(seq\)](#)

Appends the contents of seq to list

[list.index\(obj\)](#)

Returns the lowest index in list that obj appears

[list.insert\(index, obj\)](#)

Inserts object obj into list at offset index

[list.pop\(obj=list\[-1\]\)](#)

Removes and returns last object or obj from list

[list.remove\(obj\)](#)

Removes object obj from list

[list.reverse\(\)](#)

Reverses objects of list in place

[list.sort\(func\)](#)

Sorts objects of list, use compare func if given

Iterating over Lists in Python

- **For Loops**

for statement	Example use
<pre>for k in sequence: suite</pre>	<pre>nums = [10, 20, 30, 40, 50, 60] for k in nums: print(k)</pre>

- **While loop**

```
k = 0  
while k < len(nums):  
    print(nums[k])  
    k = k + 1
```

```
num=[1,2,3,4,5,6,7,8,9,10]  
for el in [1,2,3,4,5]:  
    print(el*2)  
  
2  
4  
6  
8  
10  
  
for letter in "pyhtonprogramming":  
    print(letter)  
  
p  
y  
h  
t  
o  
n  
p  
r  
o  
g
```

r
a
m
m
i
n
g

```
nums=[1,2,3,4,5,6,7,8]  
for k in [1,2,3,4]:  
    print(k)
```

```
k=0  
while k<len(nums):  
    print(nums[k])  
    k=k+1
```

1
2
3
4
1
2
3
4
5
6
7
8

You can either use loops:

```
squares = []  
for x in range(10):  
    squares.append(x**2)  
print squares  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Or you can use list comprehensions to get the same result:

```
squares = [x**2 for x in range(10)]  
print squares  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
for el in range(10):  
    print(el)
```

```
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

```
for el in range(10):  
    print(el**2)
```

```
nums=[]  
nums.append(3)
```

```

nums=[x+4 for x in range(20)]
print(nums)

[4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]

listOfWords = ["this","is","ab","list","of","words"]
items = [ word*2 for word in listOfWords]
print (items)

string = "Hello 12345 World"
numbers = [x*2 for x in string if x.isdigit()]
print (numbers)

['thisthis', 'isis', 'abab', 'listlist', 'ofof', 'wordswords']
['11', '22', '33', '44', '55']

```

Answer this?

LET'S TRY IT

From the Python Shell, enter the following and observe the results.

>>> for k in [4, 2, 3, 1]: print(k) ???	>>> for k in ['Apple', 'Banana', 'Pear']: print(k) ???
>>> for k in (4, 2, 3, 1): print(k) ???	>>> for k in 'Apple': print(k) ???

```

for k in [4,3,2,1]:
    print(k)

for k in (4,3,2,1):
    print(k)

for k in ['Apple','Banana','pear']:
    print(k)

for k in 'Apple':
    print(k)

4
3
2
1
4
3

```



```
2
1
Apple
Banana
pear
A
p
p
l
e
```

Iterating Over List Elements vs. List Index Values

- **Iterating Over List Elements**

```
nums=[10,20,30,40,50,60]
sum=0
for k in nums:
    sum=sum+k
print(sum)
```

- **List Index Values**

```
nums=[10,20,30,40,50,60]
sum=0
for k in range(len(nums)):
    sum=sum+nums[k]
print(sum)
```

```
#Iterating over List Elements
nums=[1,2,3,4,5,6,7,8]
sums=0
for k in nums:
    sums=sums+k
print(sums)
```

36

```
#Iterating over List Index
nums=[1,2,3,4,5,6,7,8]
sums=0
for k in range(len(nums)):
    sums=sums+nums[k]
print(sums)
```

36

Examples

1. Creating Lists

```
#Initialize a list
a=[1,2,3]
b=['one','two','three']
c=[1,'one','two',3]

#Print a list
print(a)
print(b)
print(c)

#Print the length of the list
#Use len() function
print(len(a))
print(len(b))
print(len(c))
```

2. List Methods

```
#Create a list of fruits
fruit=['banana','apple','cherry']

print('Initial list is')
print(fruit)

#Changing the value of a list to demonstrate mutable list
fruit[2]='coconut'

print('After changing the value at 2nd position')
print(fruit)

#Delete an element into the list
del fruit[1]

print('After deleting the value at the position 1')
print(fruit)

#Insert an element into the list
fruit.insert(2, 'pear')
print('After inserting the value at the position 2')
print(fruit)

#Append an element to the list
fruit.append('apple')
print('After appending the value to the list')
print(fruit)
```

```

#Sort a list
fruit.sort()
print('Sorted fruit list is')
print(fruit)

#Reverse a list
fruit.reverse()
print('Fruit list in the reverse order is')
print(fruit)

```

3.append vs extend

```

#Initial list
nums=[1,2,3,4,5]

print('Initial list is')
print(nums)

#Append operation in a list
#Append [6,7,8] and print the length of the list
nums.append([6,7,8])
print('After appending')
print(nums)
print(len(nums))

#Extend operation in a list
nums2=[1,2,3,4,5]
print('Initial list is')
print(nums2)
#Extend the list and print the length of the list
nums2.extend([6,7,8])
print('After extending')
print(nums2)
print(len(nums2))

```

4.List Slicing

```

#This program demonstrates list slicing

a_list=['a','b','c','d','e','f','g','h']

print('Initial list is ')
print(a_list)

print('List slicing for [1:4]')
print(a_list[1:4])

print('List slicing for [3:]')
print(a_list[3:])

```

```
print('List slicing for [:5]')
print(a_list[:5])

print('List slicing for [:]')
print(a_list[:])
```

5. Iterating Over lists

```
nums=[1,2,3,4,5]

print('Using for loop')
#Using for loop
for k in nums:
    print(k)

print('Using while loop')
#Using while loop
k=0
while k< len(nums):
    print(nums[k])
    k+=1
```

Exercises

```
1. kingdoms=['Bacteria', 'Protozoa', 'Chromista', 'Plantae', 'Fungi','Animalia']
'''
```

Write expressions that produce the following:

- The first item of kingdoms
 - The last item of kingdoms
 - The list ['Bacteria', 'Protozoa', 'Chromista']
 - The list ['Chromista', 'Plantae', 'Fungi']
 - The list ['Fungi', 'Animalia']
- ```
'''
```

2. Write a Python program to initialize the following list and display all the elements using for loop.

```
games=['CS','FIFA','Kabaddi','Hockey','Cricket']
```

Can you modify the above program where user can enter what elements should be in the list using input()???

Hint: Initialize an empty list and use append method when the value for the list is entered.

3. Write a program that sums a series of (positive) integers entered by the user.

4. Write a program that initializes a list with odd numbers and even numbers.  
#To count how many odd numbers is in a list.  
#Sum up all the even numbers in a list.

5. Write a Python program to remove duplicates from a list  
#input: a = [10,20,30,20,10,50,60,40,80,50,40]  
#Expected output:{40, 10, 80, 50, 20, 60, 30}

6. Write a Python function that takes two lists and returns True if they have at least one common member

```
#input: list1=[1,2,3,4,5], list2=[5,6,7,8,9])
#output: True
```

```
#input: list3=[1,2,3,4,5], list4=[6,7,8,9])
#output: None
```

## Tuples

- A **tuple** is an *immutable linear data structure*. Thus, in contrast to lists, once a tuple is defined, it cannot be altered.
- **The syntax for declaring** tuples in Python is given below.
  - tuples are denoted by parentheses instead of square brackets

```
nums = (10, 20, 30)
student = ('John Smith', 48, 'Computer Science', 3.42)
```

```
zoo=('python','elephant','penguin') #Parentheses are optional
print("Number of animals in zoo",len(zoo))
new_zoo=('monkey','camel',zoo)
print("Number of cages in new_zoo",len(new_zoo))
print("All animals in new_zoo",new_zoo)
print("Animals bough from old zoo are",new_zoo[2])
```

```

print("Kast animal bought from old zoo is",new_zoo[2][2])
print("Number of animals in the new zoo is",len(new_zoo)-
1+len(new_zoo[2]))
Number of animals in zoo 3
Number of cages in new_zoo 3
All animals in new_zoo ('monkey', 'camel', ('python', 'elephant',
'penguin'))
Animals bough from old zoo are ('python', 'elephant', 'penguin')
Kast animal bought from old zoo is penguin
Number of animals in the new zoo is 5

```

- The empty tuple is written as two parentheses containing nothing:

**tup1 = ()**

- To write a tuple containing a single value you have to include a comma, even though there is only one value:

**tup1 = (50,)**

- Like string indices, tuple indices start at 0, and tuples can be sliced, concatenated and so on.

```

x=(50,)
type(x)
tuple

```

## Updating Tuple: immutable

Tuples are **immutable** which means you cannot update them or change values of tuple elements. But we are able to take portions of an existing tuple to create a new tuple as follows. Following is a simple example:

```
tup1=(1,2,3)
tup2=('a','b')
tup3=tup1+tup2
print(tup3)
#Following action is not valid
tup1[0]=100

(1, 2, 3, 'a', 'b')
```

---

```


TypeError Traceback (most recent call
last)
<ipython-input-44-17ec0f5c0529> in <module>
 4 print(tup3)
 5 #Following action is not valid
----> 6 tup1[0]=100

TypeError: 'tuple' object does not support item assignment
```

## Deleting tuple elements

- Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.
- To explicitly remove an entire tuple, just use the **del statement**. This produces the error as after “del tup” tuple does not exist any more



```
tup1=(1,2,3)
print(tup1)
del tup1
print(tup1)
```

```
(1, 2, 3)
```

```


NameError Traceback (most recent call
last)
<ipython-input-45-ff8f480a730c> in <module>
 2 print(tup1)
 3 del tup1
----> 4 print(tup1)

NameError: name 'tup1' is not defined
```

## Set

### What is set in python

- A **set** is a **mutable** data type with **nonduplicate, unordered values**, providing the usual mathematical set operations.
- The syntax for declaring sets in Python is given below.

A={ 1 , 2 , 3 , 4 }

## Set methods

| Set operator         | Set A = {1,2,3}    Set B = {3,4,5,6} |               |                                                           |
|----------------------|--------------------------------------|---------------|-----------------------------------------------------------|
| membership           | 1 in A                               | True          | True if 1 is a member of set                              |
| add                  | A.add(4)                             | {1,2,3,4}     | Adds new member to set                                    |
| remove               | A.remove(2)                          | {1,3}         | Removes member from set                                   |
| union                | A   B                                | {1,2,3,4,5,6} | Set of elements in either set A or set B                  |
| intersection         | A & B                                | {3}           | Set of elements in both set A and set B                   |
| difference           | A - B                                | {1,2}         | Set of elements in set A, but not set B                   |
| symmetric difference | A ^ B                                | {1,2,4,5,6}   | Set of elements in set A or set B, but not both           |
| size                 | len(A)                               | 3             | Number of elements in set<br>(general sequence operation) |

## Set questions

For set '*s*' containing values 1, 2, 3, and set '*t*' containing 3, 4, 5, which of the following are the correct results for each given set operation?

- (a)  $s | t \rightarrow$
- (b)  $s \& t \rightarrow$
- (c)  $s - t \rightarrow$

```
s={1,2,3,4}
t={3,4,5}
print(s | t)
print(s & t)
print(s - t)
```

```

{1, 2, 3, 4, 5}
{3, 4}
{1, 2}

simple_list=[1,2,3,4,5,1,2,3,4,5,'Hi','Hi','Hello']
print(simple_list)
print(type(simple_list))
simple_list=set(simple_list)
print(simple_list)
print(type(simple_list))
simple_list=list(simple_list)
print(simple_list)
print(type(simple_list))

[1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 'Hi', 'Hi', 'Hello']
<class 'list'>
{1, 2, 3, 4, 5, 'Hello', 'Hi'}
<class 'set'>
[1, 2, 3, 4, 5, 'Hello', 'Hi']
<class 'list'>

set(simple_list)

{1, 2, 3, 4, 5, 'Hello', 'Hi'}

sample_list = [1,1,2,2,3,3,4,4,5,5]
print(type(sample_list))

<class 'list'>

sample_list = set(sample_list)
print(sample_list)
type(sample_list)
sample_list=list(sample_list)
print(sample_list)
print(type(sample_list))

{1, 2, 3, 4, 5}
[1, 2, 3, 4, 5]
<class 'list'>

sample_tup=tuple(sample_list)
print(sample_tup,type(sample_tup))

(1, 2, 3, 4, 5) <class 'tuple'>

x = ["Name", "age"]
print(x)
y={}
y = dict(x)
print(x)

```

```
['Name', 'age']
```

```


ValueError Traceback (most recent call
last)
```

```
<ipython-input-58-f80b3e404ecf> in <module>
```

```
2 print(x)
```

```
3 y={}
```

```
----> 4 y = dict(x)
```

```
5 print(x)
```

```
ValueError: dictionary update sequence element #0 has length 4; 2 is
required
```

```
y = {}
```

```
print(type(y))
```

```
<class 'dict'>
```

```
x = (1,2)
```

```
print(type(x))
```

```
<class 'tuple'>
```

## Example

```
A={1,2,3,4}
```

```
B={3,4,5,6,7,8}
```

```
print('Set A is '+str(A))
```

```
print('Set B is '+str(B))
```

```
#Find the union of A and B
```

```
print('The union of A and B is '+str(A|B))
```

```
#Find the intersection of A and B
```

```
print('The intersection of A and B is '+str(A&B))
```

```
#Find the difference A-B
```

```
print('The difference of A and B is '+str(A-B))
```

```
Set A is {1, 2, 3, 4}
```

```
Set B is {3, 4, 5, 6, 7, 8}
```

```
The union of A and B is {1, 2, 3, 4, 5, 6, 7, 8}
```

```
The intersection of A and B is {3, 4}
```

```
The difference of A and B is {1, 2}
```

## Self Test Questions

1. Which of the following sequence types is a mutable type?

- (a) strings (b) lists (c) tuples

2. Which of the following is true?

- (a) Lists and tuples are denoted by the use of square brackets.
- (b) Lists are denoted by use of square brackets and tuples are denoted by the use of parentheses.
- (c) Lists are denoted by use of parentheses and tuples are denoted by the use of square brackets.

3. Lists and tuples must each contain at least one element.  
(TRUE/FALSE)

4. For `lst = [4, 2, 9, 1]`, what is the result of the following operation, `lst.insert(2, 3)`?

- (a) [4, 2, 3, 9, 1] (b) [4, 3, 2, 9, 1] (c) [4, 2, 9, 2, 1]

```
list1=[1,2,3,4,5,6]
list1.insert(2,8)
print(list1)

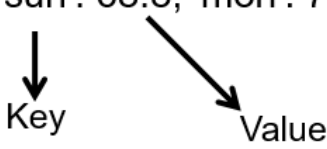
[1, 2, 8, 3, 4, 5, 6]
```

# Dictionary in python

## What is Dictionary in python?

- A **dictionary** is a **mutable**, associative data structure of variable length.
- The syntax for declaring dictionaries in Python is given below.

```
daily_temps = {'sun': 68.8, 'mon': 70.2, 'tue': 67.2, 'wed': 71.8}
```



The diagram illustrates the structure of a dictionary. It shows a dictionary declaration: `daily_temps = {'sun': 68.8, 'mon': 70.2, 'tue': 67.2, 'wed': 71.8}`. Below the code, two arrows point to the components of a dictionary item. One arrow points from the string `'sun'` to the word `Key`. Another arrow points from the float value `68.8` to the word `Value`.

- A dictionary is **mutable** and is another container type that can store any number of Python objects, including other container types.
- Dictionaries consist of **pairs (called items)** of **keys** and their corresponding **values**.
- Python dictionaries are also known as **associative arrays or hash tables**.
- **Keys are unique** within a dictionary while **values may not be**.
- The **values** of a dictionary **can be of any type**, but the **keys must be of an immutable data type** such as **strings, numbers, or tuples**.

- **The syntax for declaring dictionaries in Python** is given below.

```
daily_temps = {'sun': 68.8, 'mon': 70.2, 'tue': 67.2, 'wed': 71.8}
```

Key                      Value

- The **syntax for accessing an element of a dictionary** is the same as for accessing elements of sequence types, except that a key value is used within the square brackets instead of an index value:

```
daily_temps['sun']
```

Key

## Creating and updating Dictionary

```
dict={'Name':'Zara','Age':7,'Class':'First'}
print(dict)
#update existing dictionary
dict['Age']=8
dict['School']="DPS"
print(dict['Age'])
print(dict['School'])
print(dict)

{'Name': 'Zara', 'Age': 7, 'Class': 'First'}
8
DPS
{'Name': 'Zara', 'Age': 8, 'Class': 'First', 'School': 'DPS'}

#Deleting Dictionary elements
dict1={'Name':'Zara','Age':7,'Class':'First'}
print(dict1)
del dict1['Name']
print(dict1)
dict1.clear()
print(dict1)
```

```
{'Name': 'Zara', 'Age': 7, 'Class': 'First'}
{'Age': 7, 'Class': 'First'}
{}
```

## Properties of Dictionary elements

- **More than one entry per key not allowed.** Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins.

```
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'};
print "dict['Name']: ", dict['Name'];
```

```
dict['Name']: Manni
```

- **Keys must be immutable.** Which means you can use strings, numbers, or tuples as dictionary keys but something like ['key'] is not allowed..

```
dict = {'Name': 'Zara', 'Age': 7};
print "dict['Name']: ", dict['Name'];
```

- Dictionary Keys Are Case-Sensitive

```
dict1={'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print(dict1)
dict1={'Name': 'Zara', 'Age': 7, 'Class': 'First', 'Name': 'Minni'}
print(dict1['Name']) #last assignment wins
```

```
#keys are case sensitive
print(dict1['name'])
```

```
#keys must be immutable
dict2={'Name':'Zara','Age':7,'Class':'First'}
print(dict2['Name'])
```

```
{'Name': 'Zara', 'Age': 7, 'Class': 'First'}
Minni
```

[illegible]



```
<ipython-input-3-2b968422832e> in <module>
 5
 6 #keys are case sensitive
----> 7 print(dict1['name'])
 8
 9 #keys must be immuatble

KeyError: 'name'
```

## Dictionary methods

- [dict.items\(\)](#)
  - Returns a list of *dict*'s (key, value) tuple pairs.
- [dict.keys\(\)](#)
  - Returns list of dictionary *dict*'s keys
- [dict.values\(\)](#)
  - Returns list of dictionary *dict*'s values
- [dict.update\(dict2\)](#)
  - Adds dictionary *dict2*'s key-values pairs to *dict*

```
dict1={1:'a',2:'b',3:'c'}
print(dict1.items())
print(dict1.keys())
print(dict1.values())

dict2={4:'d',5:'e',6:'f'}
dict1.update(dict2)
print(dict2)
print(dict1)

dict_items([(1, 'a'), (2, 'b'), (3, 'c')])
dict_keys([1, 2, 3])
dict_values(['a', 'b', 'c'])
{4: 'd', 5: 'e', 6: 'f'}
{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e', 6: 'f'}
```

## fromkeys() method

- The method fromkeys() creates a new dictionary with keys from seq and values set to value.

*dict.fromkeys(seq[, value])*

- seq -- This is the list of values which would be used for dictionary keys preparation.
- value -- This is optional, if provided then value would be set to this value

```
dict1={}
seq=(1,2,3)
dict1=dict1.fromkeys(seq)
print(dict1)
dict1=dict1.fromkeys(seq,10)
print(dict1)
```

```
{1: None, 2: None, 3: None}
{1: 10, 2: 10, 3: 10}
```

```
dict2={}
lis=[1,2,3]
dict2=dict2.fromkeys(lis)
print(dict2)
```

```
{1: None, 2: None, 3: None}
```

```
dict1={'Name':"Raju",'Age':30}
print(dict1.get('Name'))
print(dict1.get('Class','Never'))
```

```
#in operator
print(1 in dict1)
```

```
Raju
Never
False
```

```
help(dict1)
```

```
Help on dict object:
```

```
class dict(object)
| dict() -> new empty dictionary
| dict(mapping) -> new dictionary initialized from a mapping
```

```

object's
| (key, value) pairs
| dict(iterable) -> new dictionary initialized as if via:
| d = {}
| for k, v in iterable:
| d[k] = v
| dict(**kwargs) -> new dictionary initialized with the name=value
pairs
| in the keyword argument list. For example: dict(one=1,
two=2)
|
| Methods defined here:
|
| __contains__(self, key, /)
| True if the dictionary has the specified key, else False.
|
| __delitem__(self, key, /)
| Delete self[key].
|
| __eq__(self, value, /)
| Return self==value.
|
| __ge__(self, value, /)
| Return self>=value.
|
| __getattr__(self, name, /)
| Return getattr(self, name).
|
| __getitem__(...)
| x.__getitem__(y) <==> x[y]
|
| __gt__(self, value, /)
| Return self>value.
|
| __init__(self, /, *args, **kwargs)
| Initialize self. See help(type(self)) for accurate signature.
|
| __iter__(self, /)
| Implement iter(self).
|
| __le__(self, value, /)
| Return self<=value.
|
| __len__(self, /)
| Return len(self).
|
| __lt__(self, value, /)
| Return self<value.

```

```

__ne__(self, value, /)
 Return self!=value.

__repr__(self, /)
 Return repr(self).

__setitem__(self, key, value, /)
 Set self[key] to value.

__sizeof__(...)
 D.__sizeof__() -> size of D in memory, in bytes

clear(...)
 D.clear() -> None. Remove all items from D.

copy(...)
 D.copy() -> a shallow copy of D

get(self, key, default=None, /)
 Return the value for key if key is in the dictionary, else
default.

items(...)
 D.items() -> a set-like object providing a view on D's items

keys(...)
 D.keys() -> a set-like object providing a view on D's keys

pop(...)
 D.pop(k[,d]) -> v, remove specified key and return the
corresponding value.
 If key is not found, d is returned if given, otherwise
KeyError is raised

popitem(...)
 D.popitem() -> (k, v), remove and return some (key, value)
pair as a
 2-tuple; but raise KeyError if D is empty.

setdefault(self, key, default=None, /)
 Insert key with a value of default if key is not in the
dictionary.

 Return the value for key if key is in the dictionary, else
default.

update(...)
 D.update([E,]**F) -> None. Update D from dict/iterable E and
F.

```

```

| If E is present and has a .keys() method, then does: for k in
E: D[k] = E[k]
| If E is present and lacks a .keys() method, then does: for k,
v in E: D[k] = v
| In either case, this is followed by: for k in F: D[k] = F[k]
|
| values(...)
| D.values() -> an object providing a view on D's values
|

| Class methods defined here:
|
| fromkeys(iterable, value=None, /) from builtins.type
| Create a new dictionary with keys from iterable and values set
to value.
|

| Static methods defined here:
|
| __new__(*args, **kwargs) from builtins.type
| Create and return a new object. See help(type) for accurate
signature.
|

| Data and other attributes defined here:
|
| __hash__ = None

```

## Example

```

scientist_to_birthdate = {'Newton' : 1642, 'Darwin' : 1809, 'Turing' :
1912}

print('All dictionary elements')
print(scientist_to_birthdate)

print('Items() method')
print(scientist_to_birthdate.items())

print('Keys() method')
print(scientist_to_birthdate.keys())

print('Get method')
print('The birthdate of Newton is '
+str(scientist_to_birthdate.get('Newton')))

```

```

#Get the birthdate for Darwin

#Add a new scientist Edison with birthdate as 1867

#Print the updated dictionary

researcher_to_birthdate = {'Curie' : 1867, 'Hopper' :
1906, 'Franklin' : 1920}

print('Before update the dictionary elements are')
print(scientist_to_birthdate)

#Updating the dictionary
scientist_to_birthdate.update(researcher_to_birthdate)

print('After update the dictionary elements are')
print(scientist_to_birthdate)

All dictionary elements
{'Newton': 1642, 'Darwin': 1809, 'Turing': 1912}
Items() method
dict_items([('Newton', 1642), ('Darwin', 1809), ('Turing', 1912)])
Keys() method
dict_keys(['Newton', 'Darwin', 'Turing'])
Get method
The birthdate of Newton is 1642
Before update the dictionary elements are
{'Newton': 1642, 'Darwin': 1809, 'Turing': 1912}
After update the dictionary elements are
{'Newton': 1642, 'Darwin': 1809, 'Turing': 1912, 'Curie': 1867,
'Hopper': 1906, 'Franklin': 1920}

```

## Exercises

1. Create a dictionary state and their capitals and print capital for the given state.

2. Write a Python script to concatenate following dictionaries to create a new one.

#Sample Dictionary :

#dic1={1:10, 2:20}

#dic2={3:30, 4:40}

#dic3={5:50,6:60}

#Expected Result : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

### 3. Phone\_book

*# This program demonstrates the use of dictionary for a phone book example.*

*# You have to write some codes as well to complete the entire working example.*

*# We have five functions*

*# 1. Create a new phone book 2. Add a new entry to the phone book*

*#3. Get the phone number of a particular person*

*#4. Get all the contacts.*

phone\_book={}

def create\_phone\_book():

*#global word\_dict*

*#word\_dict={}*

ch="y"

while (ch=="y")or(ch=="Y"):

print("\nEnter name:",end="")

name=input()

print("\nEnter number:",end="")

number=input()

*# Add a new entry to the phonebook dictionary*

*#----- Please fill the code-----*

print("\nDo you want to continue adding (y or n):",end="")

ch=input()

def add\_new\_entry():

*#global word\_dict*

print("\nEnter name:",end="")

name=input()

print("\nEnter number:",end="")

number=input()

```

 # Add a new entry to the phonebook dictionary
 #----- Please fill the code-----

def find_phone_number(name):
 print('Finding the phone number of '+name)
 # -- Please fill the code---

def get_all_contacts():
 for name,no in phone_book.items():
 print("%s ==> %s" %(name,no))

def main():
 ch="y"
 while (ch=="Y" or ch=="y"):
 print("1: Create new phone book")
 print("2: Add new name")
 print("3: Find number")
 print("4: Display phone book")
 print("5: Quit")
 print("Enter Choice: ",end="")
 option=int(input())
 if option==1:
 create_phone_book()
 elif option==2:
 add_new_entry()
 elif option==3:
 print("Enter name:",end="")
 name=input()
 print("Number of %s is :%d" %
(find_phone_number(name)))
 elif option==4:
 get_all_contacts()
 elif option==5:
 quit()

 print("\nDo you want to continue(y or n)?",end="")
 ch=input()

main()

1: Create new phone book
2: Add new name
3: Find number
4: Display phone book
5: Quit
Enter Choice:
Enter name:
Enter number:

```



```
Do you want to continue adding (y or n):
Enter name:
Enter number:
Do you want to continue adding (y or n):
Do you want to continue(y or n)?1: Create new phone book
2: Add new name
3: Find number
4: Display phone book
5: Quit
Enter Choice:
Do you want to continue(y or n)?1: Create new phone book
2: Add new name
3: Find number
4: Display phone book
5: Quit
Enter Choice:
```

## User defined Functions in python

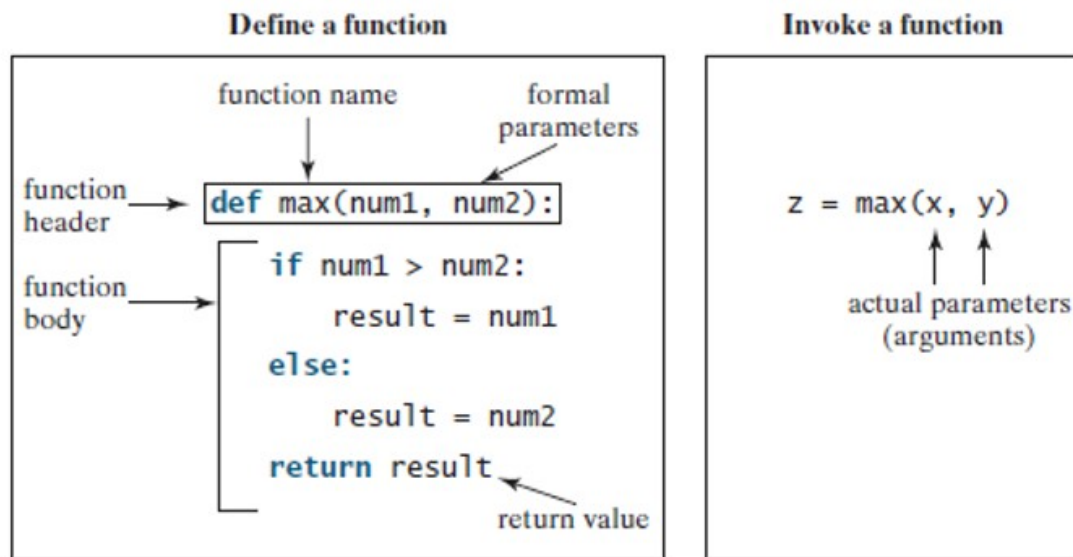
*Don't repeat code; create a function*

## Defining Functions

- “Functions can be used to define reusable code and organize and simplify code”.
- Function blocks begin with the keyword **def** followed by the function name and parentheses ( ( ) ).
- The statement **return [expression]** exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

## Syntax

```
def functionname(parameters):
 "function_docstring"
 function_suite
 return [expression]
```



**You can define a function and invoke it with arguments**

```
Return the max between two numbers
def max(num1, num2):
 """Function to find maximum of two numbers"""
 if num1 > num2:
 result = num1
 else:
 result = num2

 return result

#return the min between two numbers

def main():
 i = 5
 j = 2
 #To print the doc_string of a function use the below statement
 print(max.__doc__)
 k = max(i, j) # Call the max function
 print("The maximum between", i, "and", j, "is", k)

 #call the min function

 #print the doc_string of min function
```

```
main() # Call the main function
```

```
Function to find maximum of two numbers
The maximum between 5 and 2 is 5
```

## doc strings in python

- Python has a feature called *documentation strings*, usually referred as *docstrings*.
- **DocStrings** are an important tool that you should make use of since it helps to document the program better and makes it easier to understand.
- Docstrings provide a convenient way of associating documentation with Python modules, functions, classes, and methods.
- It's specified in source code that is used, like a comment, to document a specific segment of code.
- This allows the program to inspect these comments at run time, for instance as an interactive help system, or as metadata

```
def printMax(x,y):
 '''prints the maximum of two numbers.
```

```
The two values must be integers.'''
```

```
x=int(x)
```

```
y=int(y)
```

```
if x<y:
```

```
 print(x,"is maximum")
```

```
else:
```

```
 print(y, "is maximum")
```

```
printMax(3,5)
```

```
print(printMax.__doc__)
```

```
3 is maximum
```

```
prints the maximum of two numbers.
```

```
The two values must be integers.
```

## Scope of Variables: Local and Global Variables

```
def func(x):
 print("Local x is:",x)
 x=2
 print("Changed local x to:",x)

def func1():
 global y

 print("Y is:",y)
 y=2
 print("Changed y to:",y)

x=50
func(x)
print("X is still:",x)

y=50
func1()
print("Value of y is:",y)
```

## Function arguments

Types of formal arguments are

- Default arguments
- Keyword arguments
- Required arguments
- Variable-length arguments
- Keyword –only Parameters

## Default arguments

- The default values are passed to the parameters when a function is invoked without the arguments.
- Note that the default argument value should be immutable.
- **Note : Only those parameters which are at the end of the parameter list can be given as default argument values.**
- **For Eg. `def fun (a,b=5)→valid`  
`def fun (a=1,b) → Invalid`**

```
def printArea(width = 1, height = 2):
 area = width * height
 print("width:", width, "\theight:", height, "\tarea:", area)

print('No change in the default arguments')
printArea() # Default arguments width = 1 and height = 2

print('Default argument for change in width')
printArea(width = 1.2) # Default height = 2

print('Default argument for change in height')
printArea(height = 6.2) # Default width = 1
```

```
No change in the default arguments
width: 1 height: 2 area: 2
Default argument for change in width
width: 1.2 height: 2 area: 2.4
Default argument for change in height
width: 1 height: 6.2 area: 6.2
```

## Keyword arguments

- If you have some functions with many parameters and **you want to specify only some parameters, then you can give values for such parameters by naming them** this is called keyword arguments.
- Example: `func(c=50,a=10)`

`func(10,c=25)`

```
def func(a,b=5,c=10):
 print("a is:",a,"b is:",b,"c is:",c)
```

```
func(3,7)
func(25,c=24)
func(c=50,a=100)
```

```
a is: 3 b is: 7 c is: 10
a is: 25 b is: 5 c is: 24
a is: 100 b is: 5 c is: 50
```

## Required arguments

- If you have some functions with parameters **that has to be given while the function** is called, those parameters are called required arguments.

```
def printme(str):
 print(str)
 return

#statement below gives out the
error
#printme()

printme("Hello")
```

## Variable length arguments

- You may need to process a function for more arguments than you specified while defining the function. These arguments are called variable-length arguments

```
def functionname([formal_args,] *var_args_tuple):
 "function_docstring"
 function_suite
 return [expression]
```

- An **asterisk (\*)** is placed before the variable name that will hold the values of all non keyword variable arguments.

```
def total(initial=5,*numbers,**keywords):
 print(initial)
 print(numbers)
 print(keywords)
```



```

print('First time')
total(10,1,2,3,vegetables=50,fruits=100)
print('Second time')
total(10,1,2,vegetables=50)

#Call the function with the following parameters:
total(10,1,vegetables=50)
total(10,1,2,3,4,5,6,7,vegetables=50,fruits=20,gro=78)

```

## Keyword only parameter

- If we want to specify certain keyword parameters to be available as keyword-only and *not* as positional arguments, they can be **declared after a starred parameter**.

```

def total(initial=5,*numbers,vegetables):
 print(initial)
 print(numbers)
 print(vegetables)

total(10,1,2,3,vegetables=50)
total(10,1,2,vegetables=3)

#call the function with below given function call and notice the output
total(10,1,2,3)

```

## return statement

- The return statement is used to return from a function i.e. break out of the function. We can optionally return a value from the function as well.
- Every function **implicitly contains a return None statement**.

```

Return the grade for the score
def getGrade(score):
 if score >= 90.0:

```

```

 return 'A'
 elif score >= 80.0:
 return 'B'
 elif score >= 70.0:
 return 'C'
 elif score >= 60.0:
 return 'D'
 else:
 return 'F'

def main():
 score = eval(input("Enter a score: "))
 print("The grade is", getGrade(score))

main() # Call the main function

Enter a score: 100
The grade is A

```

## Returning multiple values

- The Python **return** statement can **return multiple values**.
- Python allows a function to return multiple values.

```

def sort(number1, number2, number3):
 if number1 < number2:
 return number1, number2, number3
 else:
 return number2, number1, number3

n1, n2, n3 = sort(3, 2, 10)
print("n1 is", n1)
print("n2 is", n2)
print("n3 is", n3)

n1 is 2
n2 is 3
n3 is 10

```

## References

- When you create an object and assign it to a variable, the variable only *refers to the object* and does not represent the object itself!
- The variable name points to that part of your computer's memory where the object is stored. This is called as **binding of the name** to the object.

```
print("Simple assignment")
shoplist=['apple','mango','carrot','banana']

#mylist is just another name pointing to the same object
mylist=shoplist

#I purchased the first item , so t remove it from the list
del shoplist[0]

print("shoplist is:",shoplist)

print("Mylist is:",mylist)
#notice that both shoplist and mylist both print the same list without
#the 'apple' confirming that they point to the same object

print("copy by making a full slice")
mylist=shoplist[:]
del mylist[0]

#Notice that now two lists are different
print("shoplist is:",shoplist)

print("Mylist is:",mylist)
```

## Pass by references vs value

- All parameters (arguments) in the Python language are passed by reference. It means if you change what a parameter refers to within a function, the change also reflects back in the calling function.

```
def changeme(mylist):
 mylist.append([1,2,3,4])
 print("Values inside the function",mylist)

mylist=[5,6,7,8]
changeme(mylist)
print("Values outside the fi=unction",mylist)

Values inside the function [5, 6, 7, 8, [1, 2, 3, 4]]
Values outside the fi=unction [5, 6, 7, 8, [1, 2, 3, 4]]

def changeme(mylist):
 mylist=[1,2,3,4]
 print("Values inside the function",mylist)

mylist=[10,20,30]
changeme(mylist)
print("Values outside the fi=unction",mylist)
```

## Exercises

1. Define a function max\_of\_three() that takes three numbers as arguments and returns the largest of them using "default arguments"

1. Define a function sum() and a function multiply() that sums and multiplies (respectively) all the numbers in a list of numbers. For example, sum([1, 2, 3, 4]) should return 10, and multiply([1, 2, 3, 4]) should return 24 using "VarArgs".

Indented block

1. Define a function `generate_n_chars()` that takes an integer `n` and a character `c` and returns a string, `n` characters long. For example, `generate_n_chars(5,"x")` should return the string `"xxxxx"` using "keyword only parameters".

## Lambda Functions in Python

# Lambda, Map, filter and reduce Functions

## Lambda Function

- Python supports the creation of anonymous functions (i.e. functions that are not bound to a name) at runtime, using a construct called "lambda".
- **Syntax: lambda arguments: expression**
- Lambda functions can have any number of arguments but only one expression. The expression is evaluated and returned.
- A simple 1-line function
- Do not use def/return keywords

```
#mx x,y
x=5
y=6

def mx(x,y):
 if x>y:
 return x
 else:
 return y

print(mx(x,y))

#Using Lambda functions

mx=lambda x,y: x if x >y else y

print(mx(x,y))

6
6
```

Functional concepts used with lambda are:

- Filter()
- Map()
- Reduce()

### map() function

- **Syntax: `r = map(func, list)`**
- The first argument *func* is the name of a function and the second a list)
- *map()* applies the function *func* to all the elements of the sequence *seq*.
- It returns a new list with the elements changed by *func*

```
#Input n=[4,3,2,1]
#print [16,9,4,1]

def square(lst1):
 lst2=[]
 for num in lst1:
 lst2.append(num **2)
 return lst2

print(square([4,3,2,1]))

#using map function
n=[4,3,2,1]
print(list(map(lambda x:x**2,n)))

#using list comprehension
print([x**2 for x in n])
```

```
[16, 9, 4, 1]
[16, 9, 4, 1]
[16, 9, 4, 1]
```

## filter() function

- **Syntax: filter(function, list)**
- The function filter(f,l) needs a function f as its first argument. f returns a Boolean value, i.e. either True or False.
- This function will be applied to every element of the list l.
- Only if f returns True will the element of the list be included in the result list.

```
#Input n=[4,3,2,1]
#print [4,3]

def over_two(lst1):
 lst2=[x for x in lst1 if x>2]
 return lst2

print(over_two([4,3,2,1]))

#using filter function
n=[4,3,2,1]
print(list(filter(lambda x:x>2,n)))

#using list comprehension
print([x for x in n if x>2])

[4, 3]
[4, 3]
[4, 3]
```



## reduce() function

- The function `reduce(func, seq)` continually applies the function `func()` to the sequence `seq`.
- It returns a single value.

```
#Input n=[4,3,2,1]
#print [4,3]

from functools import reduce
def mult(lst1):
 prod=lst1[0]
 for i in range(1,len(lst1)):
 prod*=lst1[i]
 return prod
print(mult([4,3,2,1]))

#using reduce function
n=[4,3,2,1]
print(reduce(lambda x,y:x*y,n))

108
24
```

## Exercises

1. Given a list of temperatures recorded as Celsius = [39.2, 36.5, 37.3, 37.8] , convert the readings into Fahrenheit using `map()` function and print the values

1. Given a string "Its raining cats and dogs" and perform the following operations sequentially. "Split each words of a string ,Print the length of each words of a string using `map()` function.

1. Consider a list fib = [1,1,2,3,5,8,13,21,34,55], use filter() method to print the values of the list as [2,8,34]

1. Count the number of even numbers from 1 to 1000 using filter() and lambda() functions.

1. Given a list of strings,remove all the strings that has the first character as 'h', using filter() and lambda functions

1. Given a number 'n' print the powers of it until 100 and filter out the values of the power where sum is less than 10

